2. If $S \in \mathcal{S}$, and $A \in \mathcal{A}$, then $do(A, S) \in \mathcal{S}$, where $\mathcal{A}$ is the domain of actions in the model.

- These axioms say that the tree of situations is really a tree. No cycles, no merging. It does not say that all models of these axioms have isomorphic trees (because they may have different domains of actions).
- Situations are finite sequences of actions. cf. LISP:
  - S_0 is just like NIL.
  - do acts like $cons$. $do(C, do(B, do(A, S_0)))$ is simply an alternative syntax for the LISP list $(C\ B\ A) = cons(C, cons(B, cons(A, NIL)))$.
  - To obtain the action $history$ corresponding to this term, namely the performance of action A, followed by B, followed by C, read this list from right to left.
  - Therefore, when situation terms are read from right to left, the relation $s \sqsubset s'$ means that situation s is a proper subhistory of the situation s'.
  - The situation calculus induction axiom is simply the induction principle for lists: If the empty list has property P and if, whenever list s has property P so does $cons(a, s)$, then all lists have property P.
- These 4 axioms are $domain\ independent$. They will provide the basic properties of situations in any domain specific axiomatization of particular fluents and actions.
- Henceforth, call them Σ.

Some Consequences of these Axioms

$S_0 \neq do(a, s)$.

$s = S_0 \vee (\exists a, s') s = do(a, s')$. (Existence of a predecessor)

$S_0 \sqsubseteq s$.

$s_1 \sqsubset s_2 \supset s_1 \neq s_2$. (Unique names)

$\neg s \sqsubset s$. (Anti-reflexivity)

$s \sqsubset s' \supset \neg s' \sqsubset s$. (Anti-symmetry)

$s_1 \sqsubset s_2 \wedge s_2 \sqsubset s_3 \supset s_1 \sqsubset s_3$. (Transitivity)

$s \sqsubseteq s' \wedge s' \sqsubseteq s \supset s = s'$.

The Principle of Double Induction

$$
\begin{aligned}
&(\forall R).R(S_0, S_0) \ \wedge \\
&\quad [(\forall a, s).R(s, s) \supset R(do(a, s), do(a, s))] \ \wedge \\
&\quad [(\forall a, s, s').s \sqsubseteq s' \wedge R(s, s') \supset R(s, do(a, s'))] \\
&\qquad \supset (\forall s, s').s \sqsubseteq s' \supset R(s, s').
\end{aligned}
$$

Executable Situations

- A situation is a finite sequence of actions. There are no constraints on the actions entering into such a sequence, so that it may not be possible to actually execute these actions one after the other.
- $Executable$ situations: Action histories in which it is actually possible to perform the actions one after the other.

  $s < s' \stackrel{def}{=} s \sqsubset s' \wedge (\forall a, s^*).s \sqsubset do(a, s^*) \sqsubseteq s' \supset Poss(a, s^*).$

  $s < s'$ means that s is an initial subhistory of s', and all the actions occurring between s and s' can be executed one after the other.

  $s \leq s' \stackrel{def}{=} s < s' \vee s = s',$

  $executable(s) \stackrel{def}{=} S_0 \leq s.$

More Consequences of the Axioms

$$executable(do(a,s)) \equiv executable(s) \wedge Poss(a,s),$$

$$executable(s) \equiv$$
$$s = S_0 \vee (\exists a, s').s = do(a,s') \wedge Poss(a,s') \wedge executable(s'),$$

$$executable(s') \wedge s \sqsubseteq s' \supset executable(s).$$

The Principle of Induction for Executable Situations

$$(\forall P).P(S_0) \wedge (\forall a, s)[P(s) \wedge executable(s) \wedge Poss(a,s) \supset P(do(a,s))] \supset (\forall s).executable(s) \supset P(s).$$

The Principle of Double Induction for Executable Situations

$$(\forall R).R(S_0,S_0) \wedge$$
$$[(\forall a, s).Poss(a,s) \wedge executable(s) \wedge R(s,s) \supset R(do(a,s),do(a,s))] \wedge$$
$$[(\forall a, s, s').Poss(a,s') \wedge executable(s') \wedge s \sqsubseteq s' \wedge R(s,s') \supset R(s,do(a,s')$$
$$\supset (\forall s, s').executable(s') \wedge s \sqsubseteq s' \supset R(s,s').$$

REASONING ABOUT SITUATIONS IN THE SITUATION CALCULUS

Why Prove Properties of World Situations?

- **Reasoning about systems.**

  $$(\forall s).light(s) \equiv [open(Sw_1, s) \equiv open(Sw_2, s)].$$

  This has the typical syntactic form for a proof by the simple induction axiom of the foundational axioms.

- **Planning.**

  - The standard logical account of planning views this as a theorem proving task.
  - To obtain a plan whose execution will lead to a world situation s in which the goal $G(s)$ will be true, establish that

    $$Axioms \models (\exists s).executable(s) \wedge G(s).$$

  - Sometimes we would like to establish that no plan could possibly lead to a given world situation. This is the problem of establishing that

    $$Axioms \models (\forall s).executable(s) \supset \neg G(s),$$

    i.e. that in all possible future world situations, G will be false.

Why Prove Properties (Continued)

- **Integrity constraints in database theory**
  Some background:
  - An *integrity constraint* specifies what counts as a legal database state. A property that every database state must satisfy.
    Examples:
    * Salaries are functional: No one may have two different salaries in the same database state.
    * No one's salary may decrease during the evolution of the database.
  - The concept of an integrity constraint is intimately connected with that of database *evolution*.
    No matter how the database evolves, the constraint will be true in all database futures.
    $\Longrightarrow$
    In order to make formal sense of integrity constraints, need a prior theory of database evolution.
  - How do databases change?
    One way is via predefined update *transactions*, e.g.
    * Change a person's salary to \$.
    * Register a student in a course.
  - Transactions provide the only mechanism for such state changes.

– We have a situation calculus based theory of database evolution, so use it!

– Represent integrity constraints as first order sentences, universally quantified over situations.

  * No one may have two different grades for the same course in any database state:

$$(\forall s)(\forall st, c, g, g').S_0 \leq s \wedge grade(st, c, g, s) \wedge grade(st, c, g', s) \supset g = g'.$$

  * Salaries must never decrease:

$$(\forall s, s')(\forall p, \$, \$').S_0 \leq s \wedge s \leq s' \wedge sal(p, \$, s) \wedge sal(p, \$', s') \supset \$ \leq \$'.$$

– **Constraint satisfaction defined:** A database *satisfies* an integrity constraint IC iff

$$Database \models IC.$$

Summary

- Both dynamically changing worlds, and databases evolving under update transactions may be represented in the situation calculus.
- In general, we assume given some situation calculus axiomatization, with a distinguished *initial situation* S_0.
- Objective is to prove properties true of all situations in the future of S_0.
  Examples:

$$(\forall s).light(s) \equiv [open(Sw_1, s) \equiv open(Sw_2, s)].$$

$$(\forall s, s', p, \$, \$').executable(s') \wedge s \sqsubseteq s' \wedge sal(p, \$, s) \wedge sal(p, \$', s') \supset \$ \leq \$'.$$

- These are sentences universally quantified over situations. Normally, such sentences requires induction!

Proving Properties of Situations: An Example

$$(\forall s).light(s) \equiv [open(Sw_1, s) \equiv open(Sw_2, s)].$$

- Assume this is true of the initial situation:

$$light(S_0) \equiv [open(Sw_1, S_0) \equiv open(Sw_2, S_0)].$$

- Successor state axioms for $open, light$:

$$open(sw, do(a, s)) \equiv \neg open(sw, s) \wedge a = toggle(sw) \vee open(sw, s) \wedge a \neq toggle(sw).$$

$$light(do(a, s)) \equiv \neg light(s) \wedge [a = toggle(Sw_1) \vee a = toggle(Sw_2)] \vee light(s) \wedge a \neq toggle(Sw_1) \wedge a \neq toggle(Sw_2).$$

- Simple induction principle:

$$P(S_0) \wedge [(\forall a, s).P(s) \supset P(do(a, s))] \supset (\forall s).P(s).$$

- So, take $P(s)$ to be:

$$light(s) \equiv [open(Sw_1, s) \equiv open(Sw_2, s)].$$

- QED

Proving Properties of Situations: Another Example

Salaries must never decrease:

$$(\forall s, s', p, \$, \$').executable(s') \wedge s \sqsubseteq s' \wedge sal(p, \$, s) \wedge sal(p, \$', s') \supset \$ \leq \$'.$$

- To change a person's salary, the new salary must be greater than the old:

$$Poss(change\text{-}sal(p, \$), s) \equiv (\exists \$').sal(p, \$', s) \wedge \$' < \$.$$

- Successor state axiom for sal:

$$sal(p, \$, do(a, s)) \equiv a = changeSal(p, \$) \vee sal(p, \$, s) \wedge (\forall \$')a \neq changeSal(p, \$').$$

- Initially, the relation sal is functional in its second argument:

$$sal(p, \$, S_0) \wedge sal(p, \$', S_0) \supset \$ = \$'.$$

- *Unique names axiom* for *change-sal*:

$$change\text{-}sal(p, \$) = change\text{-}sal(p', \$') \supset p = p' \wedge \$ = \$'.$$

- Double induction principle:

$$(\forall R).R(S_0, S_0) \wedge$$
$$[(\forall a, s).Poss(a, s) \wedge executable(s) \wedge R(s, s) \supset R(do(a, s), do(a, s))] \wedge$$
$$[(\forall a, s, s').Poss(a, s') \wedge executable(s') \wedge s \sqsubseteq s' \wedge R(s, s') \supset R(s, do(a,$$
$$\supset (\forall s, s').executable(s') \wedge s \sqsubseteq s' \supset R(s, s').$$

- The sentence to be proved is logically equivalent to:

$$(\forall s, s').executable(s') \wedge s \sqsubseteq s' \supset$$

$$(\forall p, \$, \$').sal(p, \$, s) \wedge sal(p, \$', s') \supset \$ \leq \$'.$$

So, take $R(s, s')$ to be:

$$(\forall p, \$, \$').sal(p, \$, s) \wedge sal(p, \$', s') \supset \$ \leq \$'.$$

- QED

BASIC THEORIES OF ACTIONS

Combining our Axioms

- Recall that Σ denotes the four foundational axioms for situations.
- We now consider some metamathematical properties of these axioms when combined with successor state and action precondition axioms, and unique names axioms for actions. Such a collection of axioms will be called a *basic theory of actions*.
- First we must be more precise about what counts as successor state and action precondition axioms.

The Uniform Formulas

- Let σ be a term of sort $situation$. A formula is $uniform\ in$ σ iff it does not mention the predicates $Poss$ or $\sqsubset$, it does not quantify over variables of sort $situation$, it does not mention equality on situations, and whenever it mentions a term of sort $situation$ in the situation argument position of a fluent, then that term is σ.